

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: K.Rama

Roll No.: A24126552087

Section: CSM-B

Date: 17-09-2026

1. TITLE

Design of a Dynamic Task Manager Webpage with a Live Task Counter using HTML, CSS, and JavaScript

2. AIM

To build a dynamic task manager web application that lets a user add, complete, and delete tasks, while a live counter tracks the number of tasks on the page, using HTML for structure, CSS (gradients, Flexbox, and animations) for styling, and JavaScript for DOM manipulation and event handling.

3. OBJECTIVE

The objectives of this experiment are:

- To link external CSS and JavaScript files to an HTML document.
- To dynamically create, append, and remove DOM elements using JavaScript.
- To keep a live counter synchronised with the number of items in a list.
- To toggle a CSS class on click to mark a task as completed.
- To use CSS keyframe animations to animate newly added list items.

4. THEORY

The Document Object Model (DOM) represents an HTML page as a tree of objects that JavaScript can read and modify. Methods like `document.createElement()` and `appendChild()` allow new elements to be created and inserted dynamically, while `remove()` detaches an element from the page. Event listeners, added using `addEventListener()`, let JavaScript respond to actions such as clicking the "Add" button, pressing Enter, clicking a task's text, or clicking its delete button. A simple counter pattern keeps the interface honest: after every change to the list (an add or a delete), a dedicated `updateCounter()` function reads `taskList.children.length` and writes it into the page, so the displayed count never has to be tracked separately from the actual DOM state. CSS keyframe animations, defined with `@keyframes` and applied via the `animation` property, let a newly created list item fade and slide into place instead of appearing abruptly. `classList.toggle()` is used to switch a "completed" class on and off a task item, and the corresponding CSS rule applies a strikethrough and a muted colour whenever that class is present.

5. ALGORITHM / PROCEDURE

1. Create `index.html` with a container `div` holding a heading, a live task counter (Tasks: `<strong id="taskCount">0</strong>`), an input field, an "Add" button, and an empty `<ul id="taskList">`.
2. Link an external `style.css` file and an external `script.js` file.
3. In `style.css`, style the page background with a linear-gradient, centre the container with Flexbox, style the input area and button, style list items, and define an `@keyframes` appear animation for new tasks.
4. In `script.js`, select the input field, add button, task list, and `taskCount` element using `getElementById()`.
5. Write an `updateCounter()` function that sets `taskCount.textContent` to `taskList.children.length`.

6. Write an addTask() function that reads and trims the input value, and shows an alert and returns if it is empty.
7. Inside addTask(), create a new containing a for the task text and a "Delete" <button>, and attach a click listener on the span to toggle a "completed" class on the .
8. Attach a click listener on the delete button to remove the from the list and call updateCounter().
9. Append the new to the task list, clear and refocus the input field, and call updateCounter().
10. Attach a click listener on the "Add" button and a keydown listener on the input field (for the Enter key) to call addTask().
11. Test the application in a browser: add several tasks, click a task to mark it completed, and delete tasks, checking that the counter updates correctly each time.

6. CODE EXPLANATION

The application is split into three files. index.html defines a card-like container with a heading, a live counter driven by an element with id "taskCount", an input field with id "taskInput", an "Add" button, and an empty <ul id="taskList"> that JavaScript populates. style.css gives the page a purple gradient background and centres a white, rounded, shadowed card on the screen using Flexbox; the input and button sit side by side using another flex container, and each list item is styled as a light grey row with rounded corners, a fade-and-slide-in @keyframes animation on appearance, and a strikethrough style for the .completed class. script.js contains the application logic: addTask() validates the input, builds a new from a (task text) and a delete <button> using document.createElement(), wires up a click listener on the span to toggle the "completed" class and a click listener on the delete button to remove the item, and finally appends the to the list. A single updateCounter() function is called after every add and delete so that the "Tasks: N" counter always reflects taskList.children.length, and event listeners on the "Add" button and the Enter key both call addTask().

7. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Dynamic Webpage</title>

  <link rel="stylesheet" href="style.css">
</head>

<body>

  <div class="container">

    <h1>My Dynamic Webpage</h1>

    <p class="counter">
      Tasks: <strong id="taskCount">0</strong>
    </p>
```

```

<div class="input-area">
  <input
    type="text"
    id="taskInput"
    placeholder="Enter a task..."
  >

  <button id="addButton">Add</button>
</div>

<ul id="taskList"></ul>

</div>

<script src="script.js"></script>

</body>
</html>
style.css
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: Arial, sans-serif;
}

body {
  min-height: 100vh;
  background: linear-gradient(135deg, #667eea, #764ba2);

  display: flex;
  justify-content: center;
  align-items: center;

  padding: 20px;
}

.container {
  width: 100%;
  max-width: 600px;

  background: white;
  padding: 30px;

  border-radius: 20px;

  box-shadow: 0 15px 40px rgba(0, 0, 0, 0.2);
}

```

```
h1 {
  text-align: center;
  color: #333;
  margin-bottom: 10px;
}

.counter {
  text-align: center;
  color: #666;
  margin-bottom: 25px;
}

.input-area {
  display: flex;
  gap: 10px;
  margin-bottom: 20px;
}

input {
  flex: 1;

  padding: 12px;

  border: 2px solid #ddd;
  border-radius: 8px;

  font-size: 16px;
  outline: none;
}

input:focus {
  border-color: #667eea;
}

button {
  border: none;

  padding: 12px 18px;

  border-radius: 8px;

  background: #667eea;
  color: white;

  cursor: pointer;
  font-size: 15px;
}
```

```

    transition: 0.2s;
}

button:hover {
    background: #5568d8;
    transform: translateY(-1px);
}

ul {
    list-style: none;
}

li {
    display: flex;
    align-items: center;
    justify-content: space-between;

    background: #f5f5f5;

    padding: 12px;
    margin-bottom: 10px;

    border-radius: 8px;

    animation: appear 0.25s ease;
}

li span {
    flex: 1;
    cursor: pointer;
}

li.completed span {
    text-decoration: line-through;
    color: #999;
}

.delete {
    background: #e74c3c;
    padding: 7px 12px;
}

.delete:hover {
    background: #c0392b;
}

@keyframes appear {
    from {

```

```

    opacity: 0;
    transform: translateY(-5px);
}

to {
    opacity: 1;
    transform: translateY(0);
}
}
script.js
// Select DOM elements
const taskInput = document.getElementById("taskInput");
const addButton = document.getElementById("addButton");
const taskList = document.getElementById("taskList");
const taskCount = document.getElementById("taskCount");

// Update task counter
function updateCounter() {
    taskCount.textContent = taskList.children.length;
}

// Add a new task
function addTask() {

    const taskText = taskInput.value.trim();

    // Check for empty input
    if (taskText === "") {
        alert("Please enter a task.");
        return;
    }

    // Create list item
    const li = document.createElement("li");

    // Create task text
    const span = document.createElement("span");

    span.textContent = taskText;

    // Create delete button
    const deleteButton = document.createElement("button");

```

```
deleteButton.textContent = "Delete";  
deleteButton.className = "delete";
```

```
// Mark task as completed  
span.addEventListener("click", function () {  
  
    li.classList.toggle("completed");  
  
});
```

```
// Delete task  
deleteButton.addEventListener("click", function () {  
  
    li.remove();  
  
    updateCounter();  
  
});
```

```
// Add elements to list item  
li.appendChild(span);  
li.appendChild(deleteButton);
```

```
// Add list item to DOM  
taskList.appendChild(li);
```

```
// Clear input  
taskInput.value = "";  
  
taskInput.focus();
```

```
// Update counter  
updateCounter();  
}
```

```
// Add task when button is clicked  
addButton.addEventListener("click", addTask);
```

```
// Add task when Enter key is pressed  
taskInput.addEventListener("keydown", function (event) {
```

```

if (event.key === "Enter") {
    addTask();
}

```

```

});

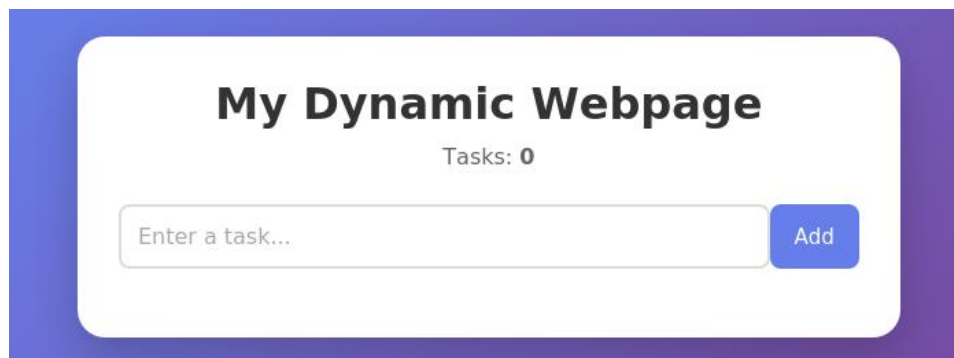
```

8. TEST CASES AND EXECUTION DATA

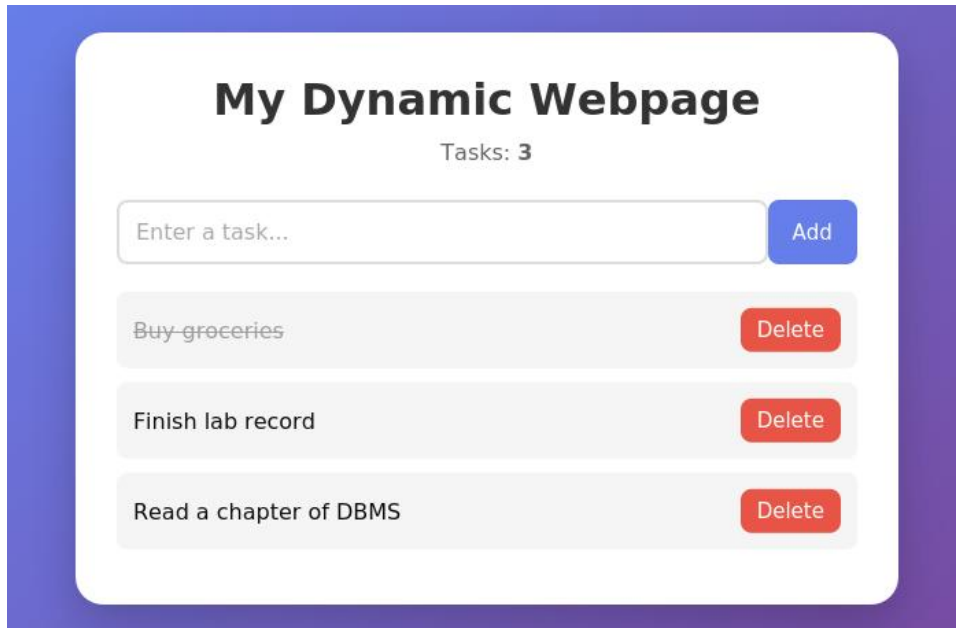
Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Load page with no tasks	Counter shows "Tasks: 0" and the list is empty	Counter and empty list displayed correctly	Pass
TC-02	Enter a task and click "Add"	New task appears in the list with a Delete button, input clears, counter increases by 1	Task added, input cleared, counter updated	Pass
TC-03	Click "Add" with empty input	Alert "Please enter a task." is shown	Alert displayed, no task added, counter unchanged	Pass
TC-04	Click on a task's text	Task text gets a strikethrough and turns grey (marked completed)	Task text shown with line-through, muted colour	Pass
TC-05	Click the "Delete" button on a task	Task is removed from the list and the counter decreases by 1	Task removed; counter updated correctly	Pass
TC-06	Press Enter key after typing a task	Task is added, same as clicking "Add"	Task added successfully via Enter key	Pass

9. OUTPUT

Initial page load (no tasks, counter at 0):



After adding three tasks and marking one as completed (counter updates to 3):



10. RESULT

Thus, a dynamic task manager web application with a live task counter was successfully designed using HTML, CSS, and JavaScript. The application allows users to add, complete, and delete tasks, the counter stays synchronised with the number of tasks at all times, and new tasks animate into view. The output was verified in a browser.